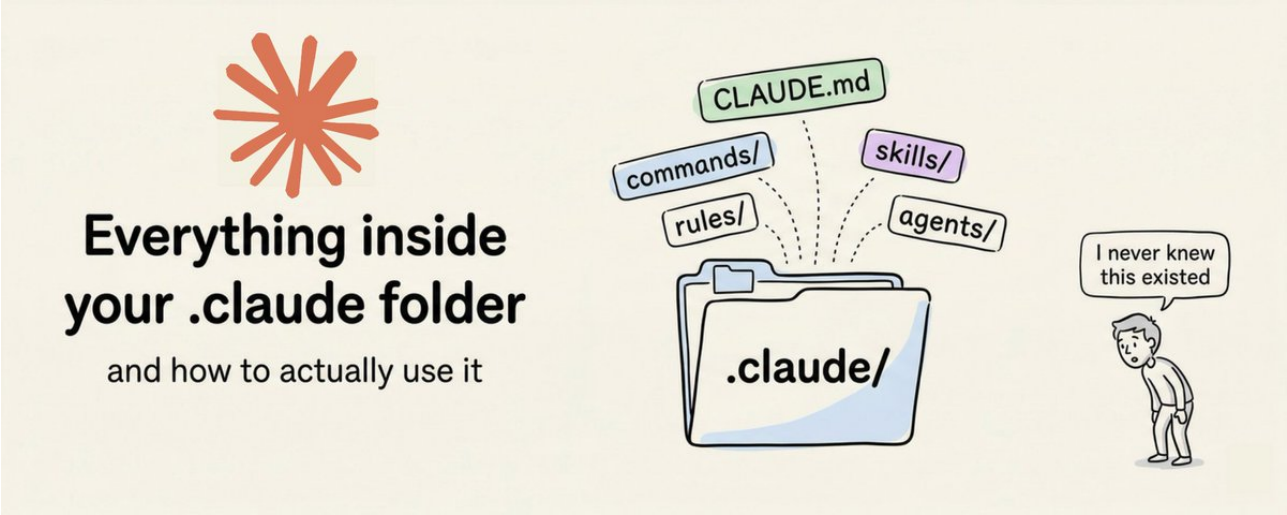
Анатомия папки .claude/

Anatomy of the .claude/ folder



Всё внутри вашей папки .claude — и как это использовать

Автор: Akshay Pachaar
https://x.com/akshay_pachaar

Оригинал: https://x.com/akshay_pachaar/status/2035341800739877091
21 марта 2026

Перевод и развёрнутые дополнения: Claude Opus 4.6 (Anthropic)
Подготовил: alienxs2 — https://github.com/alienxs2/cc_bestpractice_russian

⚠ **Развёрнутое дополнение Claude Opus 4.6 (Март 2026)**

Блоки с такой рамкой — мои развёрнутые дополнения на основе best practices Anthropic, статей Tw93, Thariq, Boris Tane, Daz и других источников на март 2026. Включают пояснения, сравнения с другими подходами, рекомендации по применению и собственные схемы/таблицы. Остальное — перевод оригинала Akshay.

Содержание

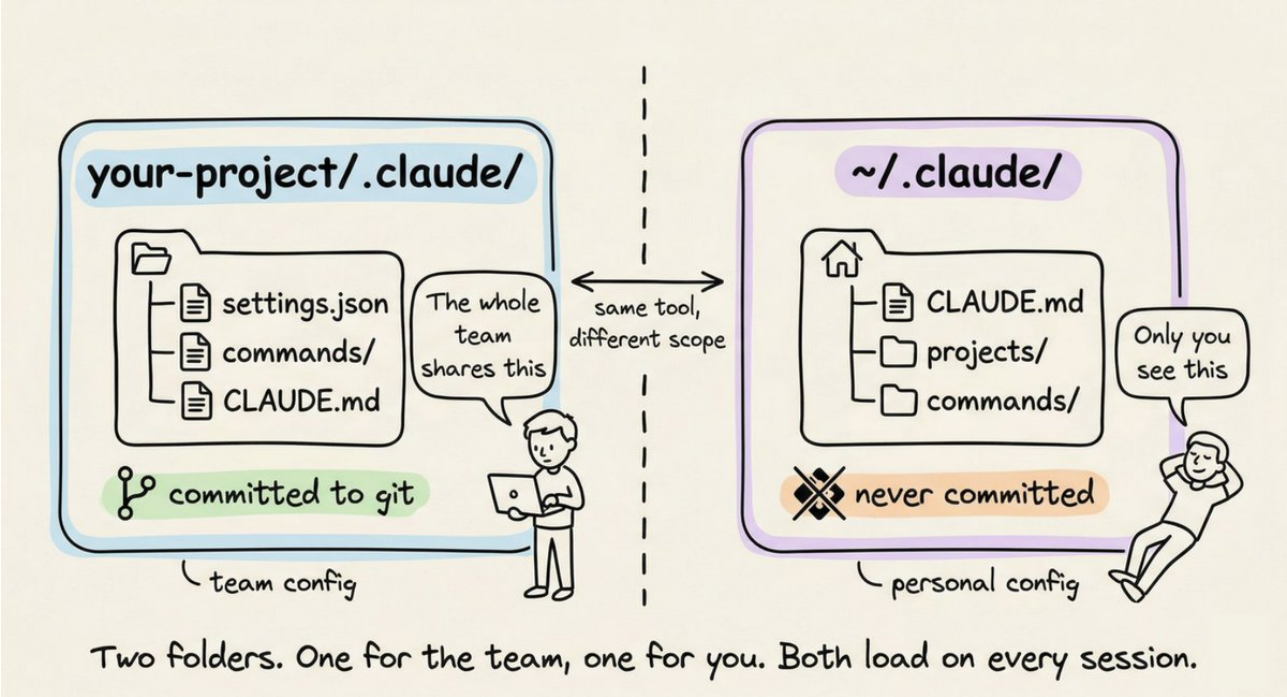
Две папки, а не одна
CLAUDE.md: инструкция для Claude
CLAUDE.local.md: личные переопределения
Папка rules/: модульные инструкции
Папка commands/: пользовательские slash-команды
Папка skills/: рабочие процессы по запросу
Папка agents/: специализированные субагенты
settings.json: разрешения и конфигурация
Глобальная папка ~/.claude/
Полная картина и план внедрения

Введение

Большинство пользователей Claude Code относятся к папке `.claude/` как к чёрному ящику. Они знают, что она существует. Видели, как она появилась в корне проекта. Но никогда не открывали, не говоря уже о понимании каждого файла внутри.

Это упущенная возможность. Папка .claude/ — это центр управления поведением Claude в вашем проекте. Она хранит инструкции, пользовательские команды, правила разрешений и даже память Claude между сессиями.

Две папки, а не одна



Две .claude/ папки: проектная (team config) и домашняя (personal config)

На самом деле существуют **две** директории .claude/

- **Проектная** `your-project/.claude/` — командная конфигурация, коммитится в git. Вся команда получает одни правила, команды, политики разрешений.
- **Домашняя** `~/.claude/` — личные предпочтения и локальное состояние: история сессий, автоматическая память. Никогда не коммитится.

⚠ **Развёрнутое дополнение Claude Opus 4.6 (Март 2026)**

Архитектурное значение двух папок: это реализация паттерна «наследование конфигурации» — аналог того, как .gitconfig работает на уровне system → global → local. В контексте multi-agent пайплайнов это означает:

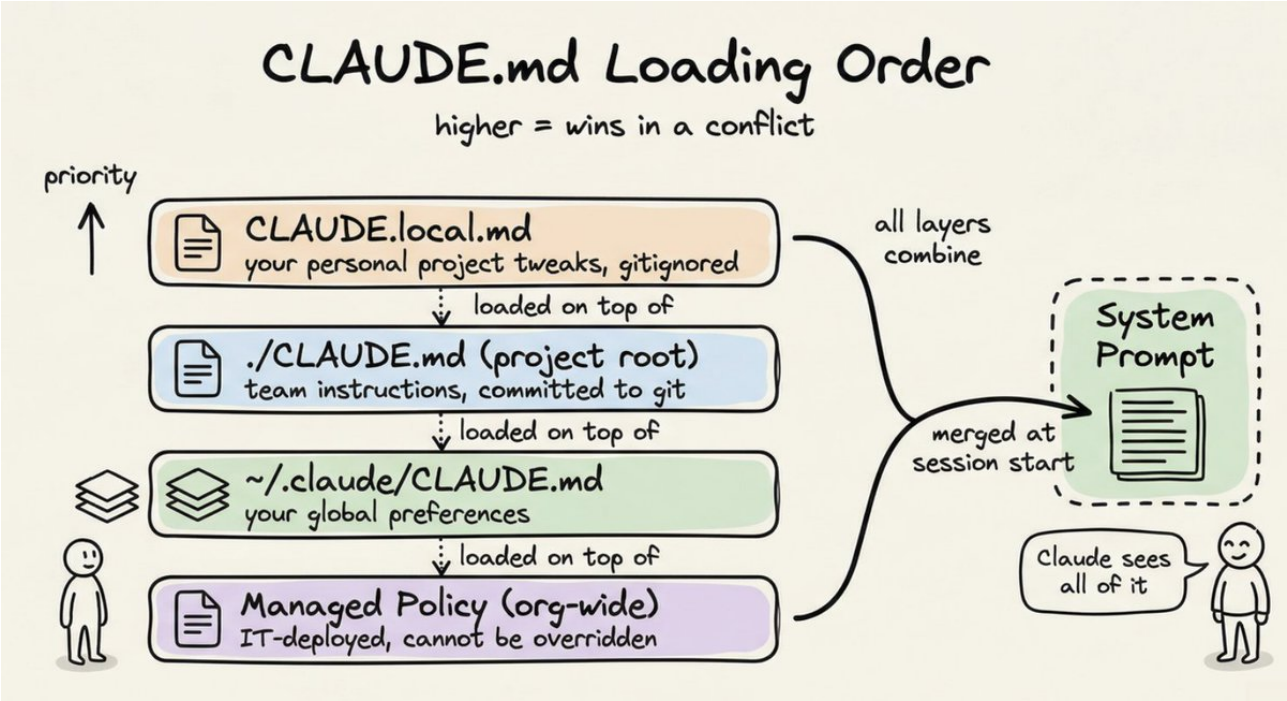
- **~/.claude/CLAUDE.md** — ваш «личный профиль PO»: стиль общения, предпочтения по объяснениям, общие принципы (например, «всегда предлагай trade-offs перед решением»)
- **project/.claude/** — контракт проекта: build-команды, архитектурные границы, NEVER-списки

Для команд с несколькими проектами рекомендация Tw93: держите стабильные личные

defaults в ~/.claude/, а проектные различия — в каждом .claude/ проекта. Это минимизирует дублирование и «загрязнение между проектами».

CLAUDE.md: инструкция для Claude

Самый важный файл во всей системе. Когда вы начинаете сессию Claude Code, первое, что он читает — CLAUDE.md. Файл загружается прямо в системный промпт и держится в памяти весь разговор.



Порядок загрузки CLAUDE.md: приоритет от низкого к высокому

Что включать:

- Команды build, test, lint (`npm run test`, `make build`)
- Ключевые архитектурные решения («monorepo с Turborepo»)
- Неочевидные gotchas («TypeScript strict mode включён, unused variables — ошибки»)
- Конвенции импортов, именования, обработки ошибок
- Структура файлов и папок основных модулей

Что НЕ включать:

- Всё, что принадлежит конфигу линтера/форматтера
- Полную документацию, на которую можно дать ссылку
- Длинные абзацы с теорией

Держите CLAUDE.md до 200 строк. Файлы длиннее начинают пожирать контекст, и adherence к инструкциям реально падает.

Минимальный, но эффективный пример:

```
# Project: Acme API
```

```
## Commands
npm run dev # Start dev server
npm run test # Run tests (Jest)
npm run lint # ESLint + Prettier check
npm run build # Production build

## Architecture
- Express REST API, Node 20
- PostgreSQL via Prisma ORM
- All handlers live in src/handlers/
- Shared types in src/types/

## Rules
- Run tests before committing
- Never modify .env files
```

~20 строк. Даёт Claude всё необходимое для продуктивной работы без постоянных уточнений.

⚠ **Развёрнутое дополнение Claude Opus 4.6 (Март 2026)**

Развёрнутый анализ: «бюджет инструкций» CLAUDE.md

Из статьи Vishwas (50 Tips): ~150-200 инструкций до падения compliance, системный промпт уже занимает ~50. Это значит, ваш CLAUDE.md может содержать реально ~100-150 действующих инструкций. Сравним подходы авторов:

- **Tw93:** CLAUDE.md = контракт. Только commands + boundaries + prohibitions. Всё остальное → rules/ и skills/
- **Thariq:** description field в Skills — это триггер «когда вызывать», а не «что внутри»
- **Boris Tane:** начни с нуля, добавляй только после повторения одной инструкции дважды
- **Daz:** детерминированное (линтеры, тесты) → hooks; недетерминированное → CLAUDE.md

Рекомендация: каждую строку в CLAUDE.md проверяйте вопросом: «Claude ошибётся без неё?» Если нет — это шум, удаляйте.

CLAUDE.local.md: личные переопределения

Иногда у вас есть предпочтение, специфичное для вас, а не для команды. Создайте `CLAUDE.local.md` в корне проекта. Claude читает его вместе с основным CLAUDE.md, и он автоматически gitignored — личные настройки не попадут в репо.

Папка rules/: модульные инструкции, которые масштабируются

CLAUDE.md отлично работает для одного проекта. Но когда команда растёт, получается 300-строчный файл, который никто не поддерживает.

rules/ решает это. Каждый markdown-файл в `.claude/rules/` загружается автоматически вместе с CLAUDE.md.

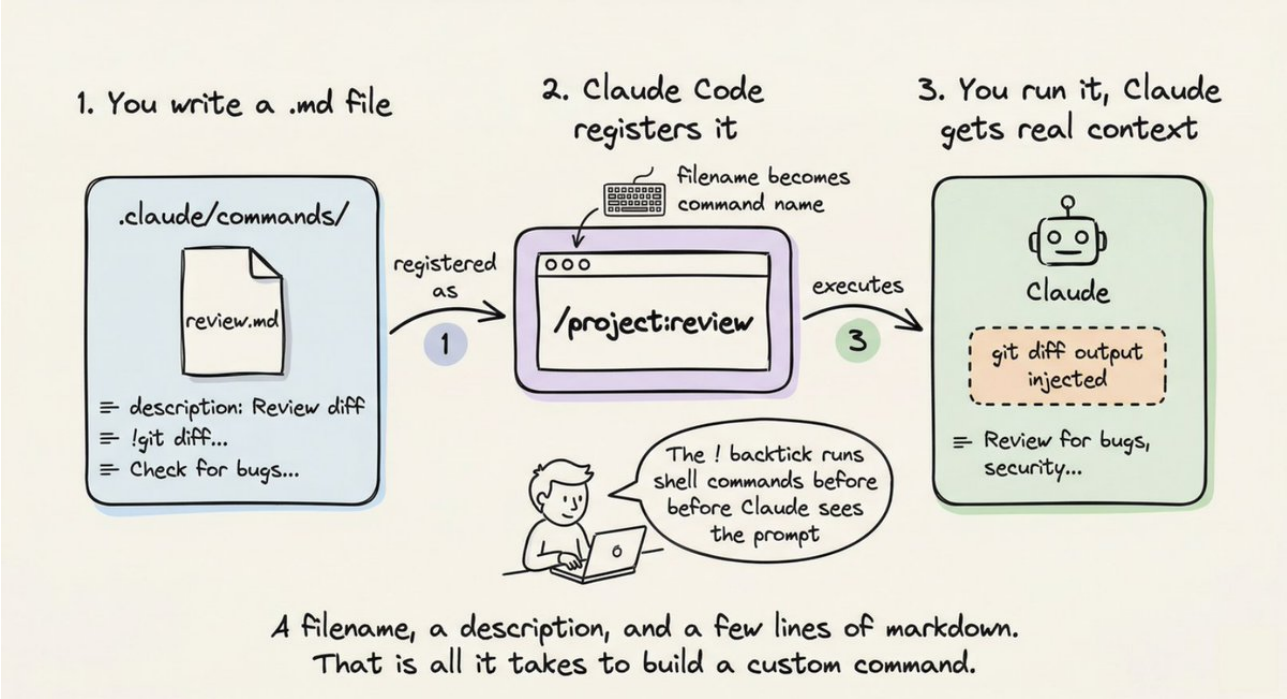
```
.claude/rules/
├── code-style.md
├── testing.md
├── api-conventions.md
```

```
└── security.md
```

Суперсила: правила по путям (path-scoped):

```
---
globs:
- "src/api/**/*.ts"
- "src/handlers/**/*.ts"
---
# API Design Rules
All handlers return { data, error } shape
Use zod for request body validation
Never expose internal error details to clients
```

Claude НЕ загрузит этот файл при редактировании React-компонента. Только при работе внутри src/api/ или src/handlers/. Правила без поля `globs` загружаются безусловно.



Папка rules/: модульные инструкции с path-scoping

⚠ **Развёрнутое дополнение Claude Opus 4.6 (Март 2026)**

Когда переходить от CLAUDE.md к rules/:

Простой тест: если ваш CLAUDE.md > 80 строк, пора разделять. Паттерн миграции:

1) Вынесите language-specific правила в rules/ с globs (TypeScript, Python, Rust отдельно)
2) Вынесите domain-specific правила (API, UI, database) в отдельные файлы
3) Оставьте в CLAUDE.md ТОЛЬКО: build-команды, архитектурные границы, NEVER/ALWAYS списки

Экономия контекста: из статьи Tw93 — типичный MCP-сервер съедает 4000-6000 токенов. Path-scoped rules не загружаются когда не нужны — это прямая экономия контекста. Для проекта с 5+ языками/доменами это может сэкономить 10-20% контекстного окна.

Папка commands/: пользовательские slash-команды

Каждый markdown-файл в `.claude/commands/` становится slash-командой. Файл `review.md` создаёт `/project:review`.

```
# .claude/commands/review.md
---
description: Review current branch diff before merging
---
## Changes to Review
`git diff --name-only main...HEAD`
`git diff main...HEAD`

Review for: code quality, security, missing tests, performance.
Give specific, actionable feedback per file.
```

Обратные кавычки (`команда`) выполняют shell-команды и встраивают вывод в промпт. Это то, что делает команды реально полезными, а не просто сохранённым текстом.

Передача аргументов:

```
# .claude/commands/fix-issue.md
---
description: Investigate and fix a GitHub issue
argument-hint: [issue-number]
---
Look at issue #$ARGUMENTS in this repo.
`gh issue view $ARGUMENTS`
Understand the bug, fix it, write a test.
```

`/project:fix-issue 234` подставит содержимое issue #234 прямо в промпт.

Проектные команды в `.claude/commands/` коммитятся и доступны всей команде. **Личные** в `~/.claude/commands/` доступны как `/user:command-name`.

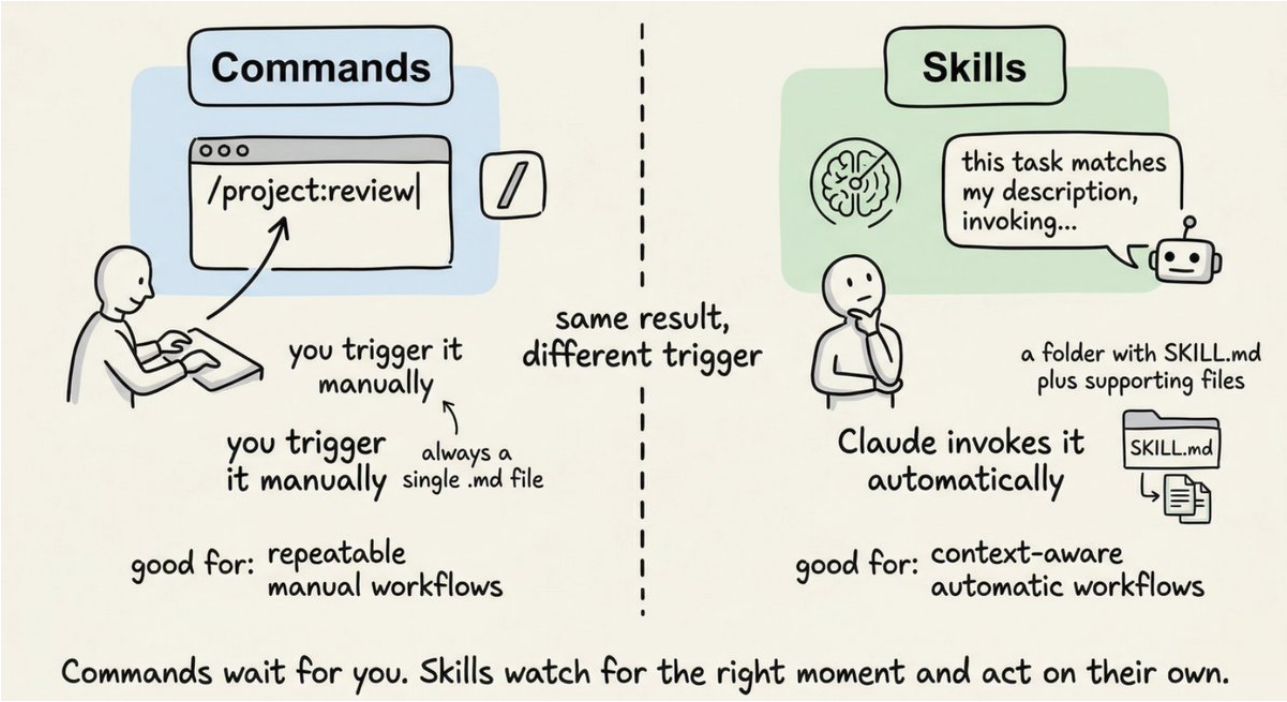
Папка skills/: рабочие процессы по запросу

Skills похожи на commands, но триггер фундаментально другой: **Skills — это workflow, которые Claude может вызвать сам**, когда задача соответствует описанию. Commands ждут вас. Skills следят за разговором и действуют когда момент подходящий.

```
.claude/skills/
├── security-review/
│ ├── SKILL.md
│ └── DETAILED_GUIDE.md
└── deploy/
├── SKILL.md
└── templates/
└── release-notes.md
```

```
# .claude/skills/security-review/SKILL.md
---
name: security-review
description: Comprehensive security audit. Use when reviewing
code for vulnerabilities or before deployments.
allowed-tools: Read, Grep, Glob
---
Analyze for: SQL injection, XSS, exposed credentials,
insecure configs, auth gaps.
Reference @DETAILED_GUIDE.md for our security standards.
```

Ключевое отличие от commands: skills могут включать поддерживающие файлы рядом с SKILL.md. Ссылка `@DETAILED_GUIDE.md` подтягивает документ из папки скилла. Commands — одиночные файлы. **Skills — пакеты.**



Commands vs Skills vs Agents — ключевые различия

⚠ **Развёрнутое дополнение Claude Opus 4.6 (Март 2026)**

Commands vs Skills vs Agents — матрица выбора:

Из статьи Thariq (Anthropic) о Skills мы знаем 9 категорий. Вот когда что использовать:

• **Command** — вы знаете КОГДА вызвать. Детерминированный триггер. Пример: /project:review, /project:fix-issue. Простой, одиночный файл.
• **Skill** — Claude решает КОГДА вызвать на основе описания. Может включать скрипты, примеры, reference-файлы. Пример: security-review, config-migration.
• **Agent** — изолированный субагент с ограниченными инструментами и своей моделью. Пример: code-reviewer (Opus, read-only), quick-search (Haiku).

Правило: если workflow вызывается >1 раз/сессию и не требует изоляции → Command. Если нужна автоматическая активация или пакет файлов → Skill. Если нужна изоляция контекста или другая модель → Agent.

Папка agents/: специализированные субагенты

Когда задача достаточно сложна для выделенного специалиста — определите субагент в `.claude/agents/`. Каждый агент — markdown-файл с собственным системным промптом, доступом к инструментам и предпочтением модели.

```
# .claude/agents/code-reviewer.md
---
name: code-reviewer
description: Expert code reviewer. Use PROACTIVELY when
reviewing PRs or validating implementations.
tools: Read, Grep, Glob
model: haiku
---
You are a senior code reviewer.
```

```
Flag bugs, not style issues.
Suggest specific fixes.
Check edge cases and error handling.
```

Поле `tools` ограничивает возможности агента. Security auditor нужны только Read, Grep, Glob — ему незачем писать файлы. Поле `model` позволяет использовать дешёвую модель для фокусных задач. Haiku хорош для read-only исследований. Opus — для работы, которая реально его требует.

⚠ **Развёрнутое дополнение Claude Opus 4.6 (Март 2026)**

Принцип least privilege для субагентов:

Из статьи Tw93: главная ценность субагентов — **изоляция**, а не параллелизм. Каждый субагент работает в своём контекстном окне. Шум исследования не загрязняет основной контекст.

Рекомендуемый набор субагентов для типичного проекта:
• **explorer** (Haiku) — быстрый поиск по файлам, Read-only
• **code-reviewer** (Opus) — ревью PR, Read + Grep + Glob
• **security-auditor** (Opus) — аудит безопасности, Read + Grep
• **test-writer** (Sonnet) — генерация тестов, Read + Write + Bash(npm test)

Из статьи Daz: субагентам не нужны те же широкие permissions, что у основного потока. Явное ограничение tools — это не ограничение, а защита вашего контекста и безопасности.

---

settings.json: разрешения и конфигурация

`.claude/settings.json` контролирует, что Claude может и не может делать:

```
{
"$schema": "https://json.schemastore.org/claude-code-settings.json",
"allowedTools": [
"Bash(npm run *)",
"Bash(git status)",
"Bash(git diff *)",
"Read", "Write", "Edit", "Glob", "Grep"
],
"deniedTools": [
"Bash(rm -rf *)",
"Read(./.env.*)"
]
}
```

- **allowedTools** — команды без подтверждения. npm run *, git *, файловые операции.
- **deniedTools** — заблокировано полностью. rm -rf, curl, .env.
- Что не в списках — Claude спросит перед выполнением.

`settings.local.json` — личные переопределения разрешений, автоматически gitignored.

⚠ **Развёрнутое дополнение Claude Opus 4.6 (Март 2026)**

Трёхзонная модель безопасности:

settings.json реализует три зоны: зелёная (allow — без подтверждения), красная (deny — заблокировано), жёлтая (всё остальное — спрашивает). Из совета #33 Vishwas и статьи Tw93:

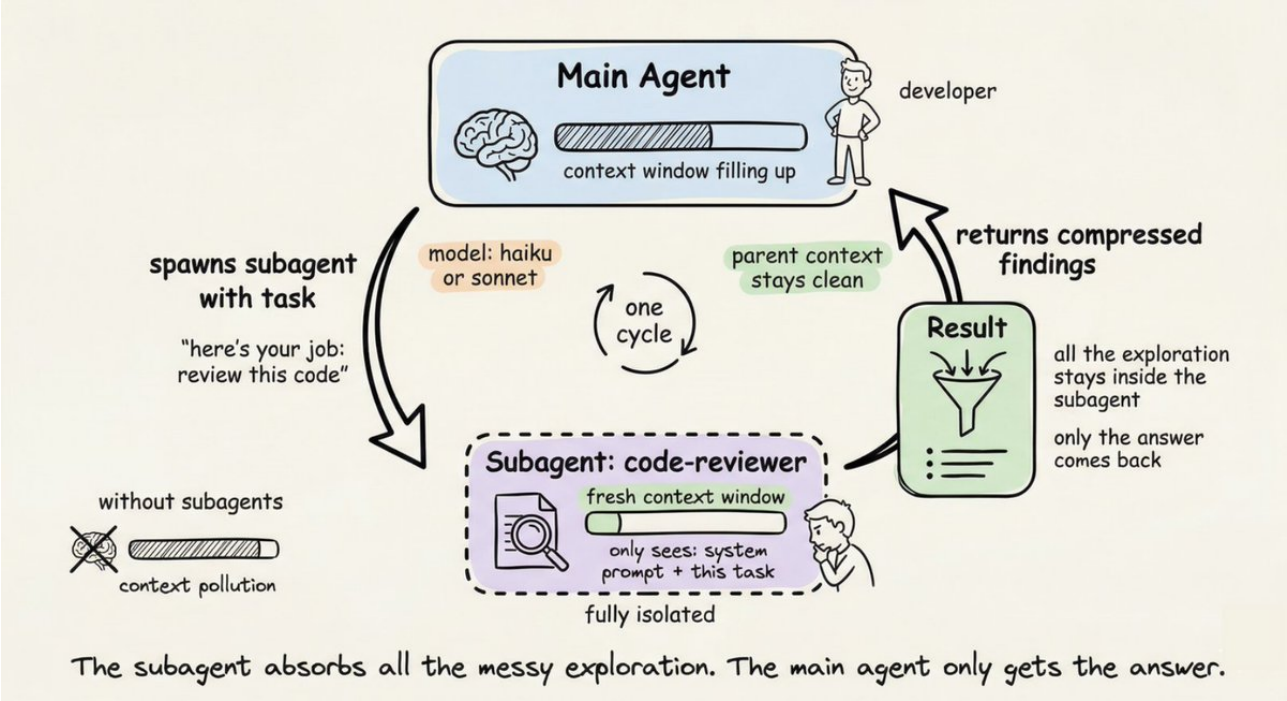
• **Зелёная зона** (allowedTools): build/test/lint команды, git read-only, файловые операции. Это то, что Claude делает постоянно — approve на каждый npm test раздражает.
• **Красная зона** (deniedTools): rm -rf, drop table, force-push, .env, secrets/. Из статьи Daz: «если машина может проверить — машина должна обеспечить».
• **Жёлтая зона** (ask): всё новое/неожиданное. Это safety net.

Критически важно: регулярно ревизуйте allowedTools! Из антипаттернов Tw93: «одобренные команды накапливаются без чистки — опасные операции вроде rm -rf остаются в settings.json; один срабатывание — и необратимо».

Глобальная папка ~/.claude/

- `~/.claude/CLAUDE.md` — загружается в каждую сессию, для всех проектов. Место для личных принципов и стиля.
- `~/.claude/projects/` — транскрипты сессий и авто-память по проектам. Claude автоматически сохраняет заметки: команды, паттерны, инсайты архитектуры. Управление: `/memory`.
- `~/.claude/commands/` и `~/.claude/skills/` — личные команды и скиллы, доступные во всех проектах.

Полная картина



Полная структура: как всё работает вместе

```
├── CLAUDE.md # Командные инструкции (committed)
├── CLAUDE.local.md # Личные переопределения (gitignored)
└── .claude/
├── settings.json # Разрешения + config (committed)
├── settings.local.json # Личные разрешения (gitignored)
```

```
├── rules/ # Модульные правила (committed)
│ ├── code-style.md
│ ├── testing.md
│ └── api-conventions.md
├── commands/ # Slash-команды (committed)
│ ├── review.md
│ └── fix-issue.md
├── skills/ # Workflow-пакеты (committed)
│ └── security-review/
│ ├── SKILL.md
│ └── DETAILED_GUIDE.md
└── agents/ # Субагенты (committed)
├── code-reviewer.md
└── security-auditor.md
```

Практический план внедрения

Если начинаете с нуля — прогрессия, которая работает:

- **Шаг 1:** Запустите `/init` внутри Claude Code. Он сгенерирует стартовый CLAUDE.md. Сократите до сути.
- **Шаг 2:** Добавьте `.claude/settings.json` с allow/deny правилами. Минимум: разрешить run-команды, запретить .env.
- **Шаг 3:** Создайте 1-2 commands для самых частых workflow (review, fix-issue).
- **Шаг 4:** Когда CLAUDE.md станет тесным — начните выносить в `.claude/rules/` с path-scoping.
- **Шаг 5:** Добавьте `~/.claude/CLAUDE.md` с личными предпочтениями.
- **Шаг 6:** Skills и agents — когда появятся повторяющиеся сложные workflow.

Ключевой инсайт

.claude/ — это протокол для объяснения Claude, кто вы, что делает проект и каким правилам следовать. Чем яснее вы это определите, тем меньше времени потратите на исправление Claude и тем больше он потратит на полезную работу.

CLAUDE.md — файл с наибольшим рычагом. Настройте его правильно первым. Всё остальное — оптимизация.

⚠ **Развёрнутое дополнение Claude Opus 4.6 (Март 2026)**

Итоговая схема приоритетов — «Пирамида конфигурации Claude Code»:

Уровень 1 (фундамент): CLAUDE.md — контракт проекта. Без этого остальное бессмысленно. ~20-50 строк.
Уровень 2: settings.json — безопасность. Allow/deny для защиты от деструктивных операций.
Уровень 3: rules/ — масштабируемость. Path-scoped правила для растущих команд.
Уровень 4: commands/ — автоматизация. Частые workflow одной командой.
Уровень 5: skills/ + agents/ — продвинутая автоматизация. Пакеты workflow + изолированные субагенты.

90% ценности — в уровнях 1-3. Не прыгайте на skills/agents, пока не настроен CLAUDE.md и settings.json. Это прямое следствие модели шести слоёв Tw93: каждый слой строится на предыдущем, и пропускать нельзя.

Оригинал: https://x.com/akshay_pachaar/status/2035341800739877091
Автор: Akshay Pachaar — https://x.com/akshay_pachaar
Подготовил перевод: alienxs2 — https://github.com/alienxs2/cc_bestpractice_russian
Перевод и развёрнутые дополнения: Claude Opus 4.6 (Anthropic), Март 2026